Lecture -Tutorial 3

MAT700 - Mathematical Methods for Data Mining,



Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector

Alex Balinsky
Cardiff School of Mathematics
Cardiff University

Link Analysis

Facts

$$\mathbf{v} = \left((1-\beta)\frac{\mathbf{E}}{n} + \beta M \right) \mathbf{v} \tag{1}$$

$$\mathbf{v} = (1-\beta)\mathbf{e}/n + \beta M\mathbf{v} \tag{2}$$

Alex Balinsky



Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector

Link Analysis

Alex Balinsky



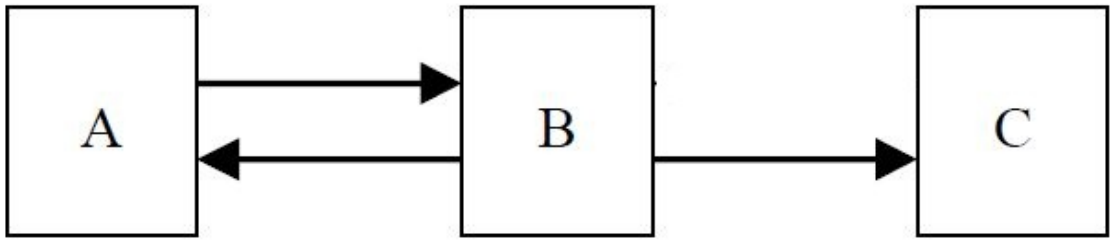
Facts

$$\mathbf{v} = \left((1-\beta)\frac{\mathbf{E}}{n} + \beta M\right)\mathbf{v} \tag{1}$$

$$\mathbf{v} = (1-\beta)\mathbf{e}/n + \beta M\mathbf{v} \tag{2}$$

Problem 1.

Calculate M for



Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector

Link Analysis

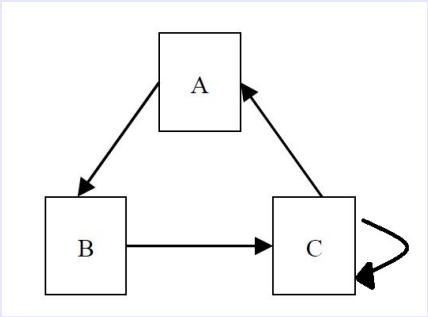
Alex Balinsky



Damping factor $\beta = 0.8$

Problem 2.

Calculate PageRank for

- 

Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector

Link Analysis

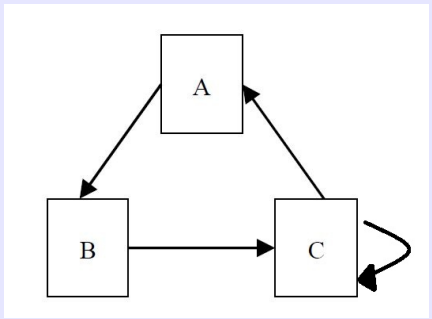
Alex Balinsky

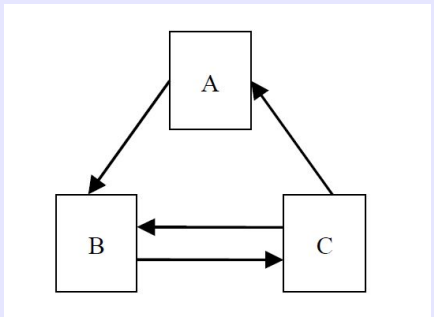


Damping factor $\beta = 0.8$

Problem 2.

Calculate PageRank for

- 

- 

Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector

Naïve Bayesian Classification



Link Analysis
Naïve Bayesian Classification
How to Write a Spelling Corrector

Problem 1.

Given the following data:

Instance	A	B	C	Class
1	1	2	1	1
2	0	0	1	1
3	2	1	2	2
4	1	2	1	2
5	0	1	2	1
6	2	2	2	2
7	1	0	1	1

Predict the class label for instance $(A = 1; B = 2; C = 2)$ using naïve Bayes classification.

Problem 2.

Given the following data:

outlook	temperature	humidity	windy	play
sunny	hot	high	false	no
sunny	hot	high	true	no
overcast	hot	high	false	yes
rainy	mild	high	false	yes
rainy	cool	normal	false	yes
rainy	cool	normal	true	no
overcast	cool	normal	true	yes
sunny	mild	high	false	no
sunny	cool	normal	false	yes
rainy	mild	normal	false	yes
sunny	mild	normal	true	yes
overcast	mild	high	true	yes
overcast	hot	normal	false	yes
rainy	mild	high	true	no

In this dataset, there are five categorical attributes *outlook, temperature, humidity, windy, and play.* We are interested in building a system to decide whether or not to play the game on the basis of the weather conditions, i.e. we wish to classify the data into two classes, one where the attribute play has the value C_1 ="yes", and the other where it has the value C_2 ="no".



Link Analysis
Naïve Bayesian Classification
How to Write a Spelling Corrector

Naïve Bayesian Classification



Link Analysis
Naïve Bayesian Classification
How to Write a Spelling Corrector

Problem 2. cont.

(1) Calculate $P(C1|X = (sunny, hot, high, false))$. How would the Naïve Bayes classifier classify the data instance $X = (sunny, hot, high, false)$?

(2) Does this agree with the classification given in Table for the data instance $X = (sunny, hot, high, false)$?

(3) Consider a new data instance $X' = (overcast, cool, high, true)$. How would the Naïve Bayes classifier classify X'?

How to Write a Spelling Corrector



The full details of an industrial-strength spell corrector are quite complex.

Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector

How to Write a Spelling Corrector



The full details of an industrial-strength spell corrector are quite complex.

What I wanted to do here is to present, in less than a page of code, a **toy spelling corrector** that achieves 90% accuracy at a processing speed of at least 10 words per second.

Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector

How to Write a Spelling Corrector



Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector

The full details of an industrial-strength spell corrector are quite complex.

What I wanted to do here is to present, in less than a page of code, a **toy spelling corrector** that achieves 90% accuracy at a processing speed of at least 10 words per second.

The code defines the function **correct**, which takes a word as input and returns a likely correction of that word. For example:

```
>>> correct('speling')
'spelling'
>>> correct('korrecter')
'corrector'
```

How to Write a Spelling Corrector

Alex Balinsky



Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector

File Edit Format Run Options Windows Help

```
"""Spelling Corrector.

Copyright 2007 Peter Norvig.
Open source code under MIT license: http://www.opensource.org/licenses/mit-license.php
"""

import re, collections

def words(text): return re.findall('[a-z]+', text.lower())

def train(features):
    model = collections.defaultdict(lambda: 1)
    for f in features:
        model[f] += 1
    return model

NWORDS = train(words(file('big.txt').read()))

alphabet = 'abcdefghijklmnopqrstuvwxyz'

def edits1(word):
   s = [(word[:i], word[i:]) for i in range(len(word) + 1)]
   deletes    = [a + b[1:] for a, b in s if b]
   transposes = [a + b[1] + b[0] + b[2:] for a, b in s if len(b)>1]
   replaces   = [a + c + b[1:] for a, b in s for c in alphabet if b]
   inserts    = [a + c + b     for a, b in s for c in alphabet]
   return set(deletes + transposes + replaces + inserts)

def known_edits2(word):
    return set(e2 for e1 in edits1(word) for e2 in edits1(e1) if e2 in NWORDS)

def known(words): return set(w for w in words if w in NWORDS)

def correct(word):
    candidates = known([word]) or known(edits1(word)) or known_edits2(word) or [word]
    return max(candidates, key=NWORDS.get)
```

How It Works: Some Probability Theory



Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector

How does it work?

First, a little theory. Given a word, we are trying to choose the most likely spelling correction for that word (the "correction" may be the original word itself).

How It Works: Some Probability Theory



Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector

How does it work?
First, a little theory. Given a word, we are trying to choose the most likely spelling correction for that word (the "correction" may be the original word itself).

There is no way **to know for sure** (for example, should "lates" be corrected to "late" or "latest"?), which suggests we use probabilities.

How It Works: Some Probability Theory



Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector

How does it work?
First, a little theory. Given a word, we are trying to choose the most likely spelling correction for that word (the "correction" may be the original word itself).

There is no way **to know for sure** (for example, should "lates" be corrected to "late" or "latest"?), which suggests we use probabilities.

We will say that we are trying to find the correction c, out of all possible corrections, that maximizes the probability of c given the original word w:

$$argmax_c P(c|w)$$

How It Works: Some Probability Theory

By **Bayes' Theorem** this is equivalent to:

$$argmax_c \frac{P(w|c) \cdot P(c)}{P(w)}$$



Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector

How It Works: Some Probability Theory



By **Bayes' Theorem** this is equivalent to:

$$argmax_c \frac{P(w|c) \cdot P(c)}{P(w)}$$

Since $P(w)$ is the same for every possible c, we can ignore it, giving:

$$argmax_c P(w|c) \cdot P(c)$$

Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector

How It Works: Some Probability Theory



Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector

By **Bayes' Theorem** this is equivalent to:

$$argmax_c \frac{P(w|c) \cdot P(c)}{P(w)}$$

Since $P(w)$ is the same for every possible c, we can ignore it, giving:

$$argmax_c P(w|c) \cdot P(c)$$

There are three parts of this expression. From right to left, we have:
1. $P(c)$, the probability that a proposed correction c stands on its own.
This is called **the language model**: think of it as answering the question "how likely is c to appear in an English text?" So P("the") would have a relatively high probability, while P("zxzxzxzyyy") would be near zero.

How It Works: Some Probability Theory



2. $P(w|c)$, the probability that w would be typed in a text when the author meant c.

This is **the error model**: think of it as answering "how likely is it that the author would type w by mistake when c was intended?"

Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector

How It Works: Some Probability Theory



Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector

2. $P(w|c)$, the probability that w would be typed in a text when the author meant c.

This is **the error model**: think of it as answering "how likely is it that the author would type w by mistake when c was intended?"

3. $argmax_c$, the control mechanism, which says to enumerate all feasible values of c, and then choose the one that gives the best combined probability score.

How It Works: Some Probability Theory

One obvious question is: why take a simple expression like $P(c|w)$ and replace it with a more complex expression involving two models rather than one?



Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector

How It Works: Some Probability Theory



Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector

One obvious question is: why take a simple expression like $P(c|w)$ and replace it with a more complex expression involving two models rather than one?

The answer is that $P(c|w)$ is already conflating two factors, and it is easier to separate the two out and deal with them explicitly.

Consider the misspelled word *w="thew"* and the two candidate corrections *c="the"* and *c="thaw"*.

Which has a higher $P(c|w)$?

How It Works: Some Probability Theory



Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector

Well, *"thaw"* seems good because the only change is *"a"* to *"e"*, which is a small change.

On the other hand, *"the"* seems good because *"the"* is a very common word, and perhaps the typist's finger slipped off the *"e"* onto the *"w"*.

The point is that to estimate $P(c|w)$ we have to consider both the probability of c and the probability of the change from c to w anyway, so it is cleaner to formally separate the two factors.

Now we are ready to show how the program works.



Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector



Now we are ready to show how the program works.

First P(c): We will read a **big text file**, *big.txt*, which consists of about a million words.

The file is a concatenation of several public domain books from *Project Gutenberg* and lists of most frequent words from *Wiktionary* and the *British National Corpus*.

Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector



Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector

Now we are ready to show how the program works.

First P(c): We will read a **big text file**, *big.txt*, which consists of about a million words.

The file is a concatenation of several public domain books from *Project Gutenberg* and lists of most frequent words from *Wiktionary* and the *British National Corpus*.

We then extract the individual words from the file (using the function *words*, which *converts everything to lowercase*, so that *"the"* and *"The"* will be the same and then defines a word as a sequence of alphabetic characters, so *"don't"* will be seen as the two words *"don"* and *"t"*).



Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector

Now we are ready to show how the program works.

First P(c): We will read a **big text file**, *big.txt*, which consists of about a million words.

The file is a concatenation of several public domain books from *Project Gutenberg* and lists of most frequent words from *Wiktionary* and the *British National Corpus*.

We then extract the individual words from the file (using the function *words*, which *converts everything to lowercase*, so that *"the"* and *"The"* will be the same and then defines a word as a sequence of alphabetic characters, so *"don't"* will be seen as the two words *"don"* and *"t"*).

Next we train a probability model, which is a fancy way of saying we count how many times each word occurs, using the function *train*.

Now let's look at the problem of enumerating the possible corrections c of a given word w.



Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector



Now let's look at the problem of enumerating the possible corrections c of a given word w.

It is common to talk of the **edit distance** between two words: the number of edits it would take to turn one into the other.

Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector



Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector

Now let's look at the problem of enumerating the possible corrections c of a given word w.

It is common to talk of the **edit distance** between two words: the number of edits it would take to turn one into the other.

An edit can be a **deletion** (remove one letter), a **transposition** (swap adjacent letters), an **alteration** (change one letter to another) or an **insertion** (add a letter).

edits1 is a function that returns a set of all words *c* that are one edit away from *w*.



Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector

Now let's look at the problem of enumerating the possible corrections c of a given word w.

It is common to talk of the **edit distance** between two words: the number of edits it would take to turn one into the other.

An edit can be a **deletion** (remove one letter), a **transposition** (swap adjacent letters), an **alteration** (change one letter to another) or an **insertion** (add a letter).

edits1 is a function that returns a set of all words *c* that are one edit away from *w*.

This can be a big set. For a word of length n, there will be n deletions, $n-1$ transpositions, $26 \cdot n$ alterations, and $26(n+1)$ insertions, for a total of $54 \cdot n + 25$ (of which a few are typically duplicates).
For example, *len(edits1('something'))* – that is, the number of elements in the result of *edits1('something')* – is 494.

The literature on spelling correction claims that 80% to 95% of spelling errors are an edit distance of 1 from the target.



Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector

The literature on spelling correction claims that 80% to 95% of spelling errors are an edit distance of 1 from the target.

This is not good enough, so we'll need to consider edit distance 2.

That's easy: just apply *edits1* to all the results of *edits1*.



Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector

The literature on spelling correction claims that 80% to 95% of spelling errors are an edit distance of 1 from the target.

This is not good enough, so we'll need to consider edit distance 2.

That's easy: just apply *edits1* to all the results of *edits1*.

This is easy to write, but we're starting to get into some serious computation: *len(edits2('something'))* is $114,324$.



Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector

The literature on spelling correction claims that 80% to 95% of spelling errors are an edit distance of 1 from the target.

This is not good enough, so we'll need to consider edit distance 2.

That's easy: just apply *edits1* to all the results of *edits1*.

This is easy to write, but we're starting to get into some serious computation: *len(edits2('something'))* is 114,324.

Since we aren't going beyond edit distance 2, we can do a small optimization: only keep the candidates that are actually known words. We still have to consider all the possibilities, but we don't have to build up a big set of them. The function *known_edits2* does this.

Now, for example, *known_edits2('something')* is a set of just 4 words: *'smoothing', 'seething', 'something', 'soothing'*, rather than the set of 114,324 words generated by *edits2*. That speeds things up by about 10%.



Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector



Now the only part left is the **error model**: $P(w|c)$.

Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector



Now the only part left is the **error model**: $P(w|c)$.

Here's where we ran into difficulty. We need a training data to build a model of spelling errors.

Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector



Now the only part left is the **error model**: $P(w|c)$.

Here's where we ran into difficulty. We need a training data to build a model of spelling errors.

We had some intuitions: mistaking one vowel for another is more probable than mistaking two consonants; making an error on the first letter of a word is less probable, etc. But we need numbers to back that up.

Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector



Let us take a *shortcut*:
We defined a trivial model that says all known words of edit distance 1 are infinitely more probable than known words of edit distance 2, and infinitely less probable than a known word of edit distance 0. By *"known word"* I mean a word that we have seen in the language model training data – a word in the dictionary.

Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector



Link Analysis

Naïve Bayesian Classification

How to Write a Spelling Corrector

Let us take a *shortcut*:
We defined a trivial model that says all known words of edit distance 1 are infinitely more probable than known words of edit distance 2, and infinitely less probable than a known word of edit distance 0. By *"known word"* I mean a word that we have seen in the language model training data – a word in the dictionary.

The function *correct* chooses as the set of candidate words the set with the shortest edit distance to the original word, as long as the set has some known words. Once it identifies the candidate set to consider, it chooses the element with the highest $P(c)$ value, as estimated by the NWORDS model.